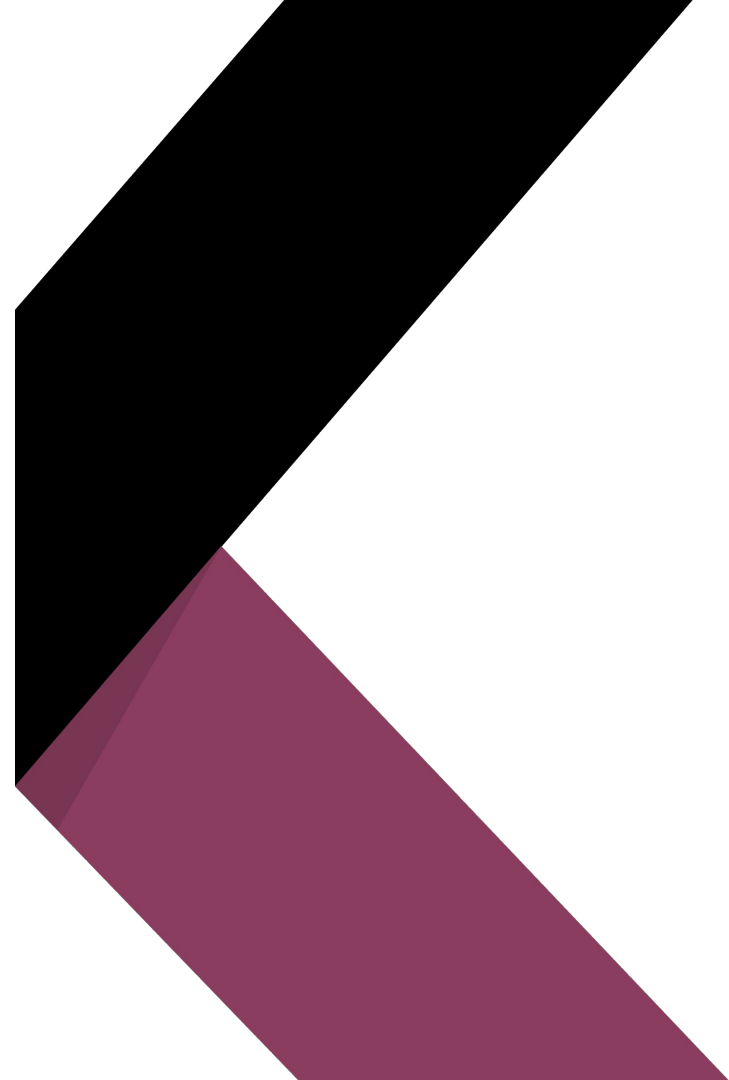


Vehicle
Integrated Functionality

Validation



Vehicle Integrated Functionalities Validation

Introduction

Why

Introduction:

The validation takes part in several development engineering process, its rule is to define the correctness of a creation compared to its original concept design.

Why do we validate?

In this case we consider the SW developing. The validation has the purpose to establish if the implemented SW logics in the development process reflects the specification requirements and if the behaviour of the control system meets the expected response.

Why in vehicle?

The vehicle integrated functionalities validation (VIF) has the purpose of testing the sw functionalities in a situation where the control is applied on complex dynamics and there is the interaction between non-simulated ECUs , and in general a real response by the controlled system is required. This test environment represent the highest qualitative analysis process of an automotive integrated system : the test are performed in a real controlled system where the answer to a specific stimulus contains all the dynamics/disturbance that a simulated environment can not provide due to high complexity in terms of elaboration.

Vehicle Integrated Functionalities Validation

Introduction

Real vs Virtual

What are the vehicle validation's advantages compared to simulated environment?

As we said, the vehicle testing validation does include real system answer to a specific stimulus. A validator can exclude all the problems caused by the inaccuracy or the limitation of the simulated environment that could negatively affect test results . At the same time, real application does represent a constraint: it is not possible to force the system in a specific condition so easily as a simulated environment where all the involved input are arbitrarily chosen by the user. In this case a specific maneuver (or a sequence) must be applied in order to let the system reach that status (e.g. the thermal engine increment – warm up). Some test forces the validator to adopt complex strategies in order reaching that required inputs : sensors are real, thus the read data is defined by a real physical entity.

On the other hand, vehicle validation can prove the simulated system robustness: real testing environment can confirm the stressed system's result performed either in a simulated environment in order to prove the correctness and reliability of the simulation itself.

Vehicle Integrated Functionalities Validation

Introduction

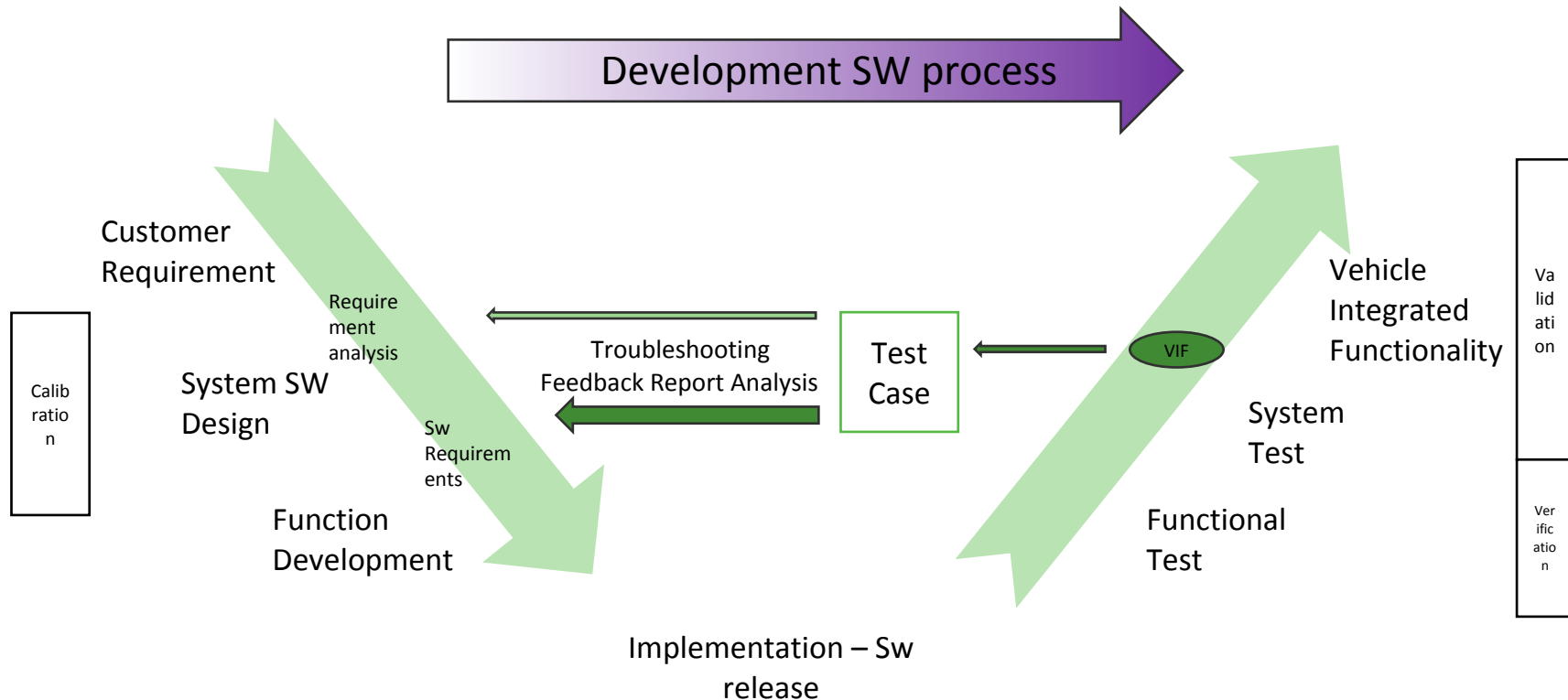
When VIF?

When is the Vehicle validation performed?

Integrated functionality is the last part of the sw investigation to be performed. Thus vehicle validation usually is performed in the last stage of the development.



V-Cycle





Aim

Following the V model , there is a progression in term of HW test environment: from open\closed loop model to the final product .

Vehicle validation process inside the V model occupies the last step of the sw development . A validator has the duty to check the reliability of the controls and the correct implementation of the sw functionality.

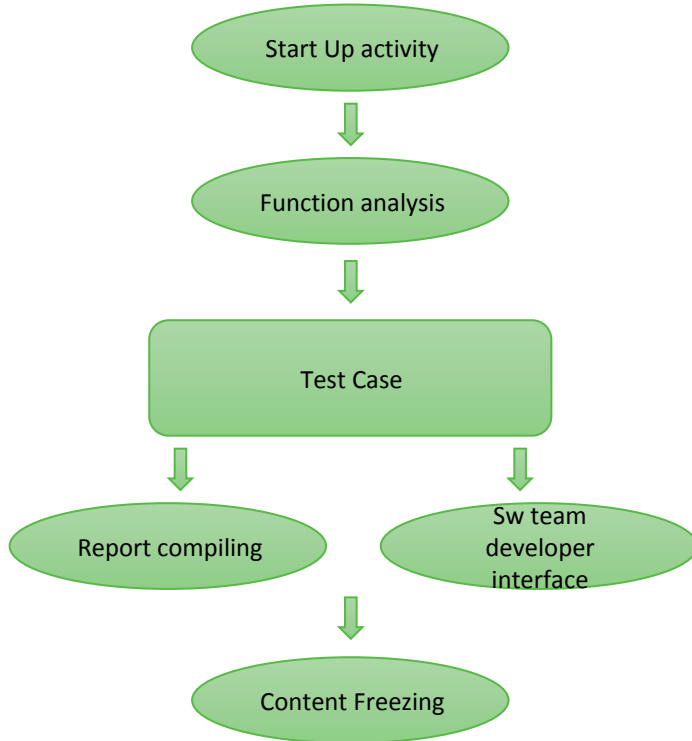
- The Val is performed following a list of test cases extracted from the requirement functionality description: CheckList , DVP (Design Validation Plan).
- System behaviour is measured and analyzed
- As final result, the validator gives feedback through analysis report of the functionality tested compared to the system requirements to the suppliers or the involved development sw team (Calibrators , Functionists ...) . In this way a feedback process inside the V-model is established , introducing where needed bugfix suggestions and sw modification (Troubleshooting).



HOW do we validate?



Vehicle validation process



- Activity plan – Test needed documentation retrieving – Car start-up (ECM sw flashing with last SW release – related interactive nodes updated the closest to the production intent)
- Function analysis : sw manual description and spec. requirements learning
- In vehicle test
- Opening traceability (DVPM) points in case of negative behaviour result confirmed by Project supervisor - Issue state monitoring
- Report compiling in order to define the Test result
- SW content freezing in case of positive test result with the collection of the calibration set.



Design Validation Plan & Results

As we previously said, the validation provides a comparison between the functional concept and the resulting SW implementation.

This process is usually prepared reporting all the specs inside a document where they are translated into a list of maneuver to be executed in vehicle; the test description must contain all relevant information for the test execution adapted in the environment where it's applied and together fulfill the requirement concept.

e.g. requirement reports that the thermostatic engine coolant valve must be closed in very first phase of engine warm up, and that ECM starts to command the opening after that the engine coolant temperature is close to the target idling temp → test case: Turn on the engine and wait for the regimated coolant temperature between 80° or 90° (depending on engine thermal characteristic), check that thermostatic valve sensed position is fully close for 10 seconds after that the engine has started and it is partially/fully opened when the engine temp has been regimated.

Once DVP is ready, the validator analyze the behaviour of the system tested and gives Result.



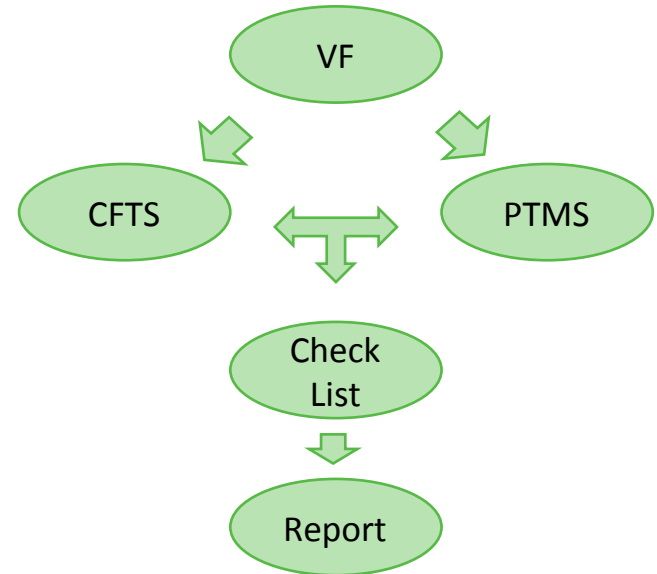


Function analysis – Stellantis Validation documentation

- *VF* (Vehicle Function) , similary to the CFTS it contains the requirement description for only one application .
- *CFTS* (Component Functional Technical Specification) , it contains the functional characteristic (e. g. speed limiter minimal set point) , and the functional structure of the engine controls in comply to the requirements to be implemented for every architecture (equivalent document) .
- *PTMS* , it collects all the specification of the vehicle interface function for engine control system . High level description of the main ECM functionalities and their interaction .
- *CheckList* , is the output documentation , created in compliance of the system requirements :

Every test present in the checklist is meant to analyze the behaviour of a function providing the expected results and negate one (if the functionality shall not be working\present) coherently with the requirements .

In some cases there could be a functionality characterized for a specific project , for example the *whole ECM speed limiter*; In this case there is dedicated checklist for the function treated .
- *General Report* , it collects all the tests results and the notes about the procedure





CheckList Layout

The checklist is the output documentation , created in compliance of the system requirements .

Indeed it reports a list of maneuvers to be applied on the system in order to validate the function . The condition represents the maneuver to be realized , while the system expected behaviour is described on the verification column (as expressed in the requirements reported on the same line) .

Defines the functionality involved

Indicates the feature of the function we are going to test

ID	Test Function Name	Sub-Function	Test Env	Tools	Condition	Verification	E/E Architecture	CFTS ID
T011_1	1 Active Speed Limiter		Veh Sim Bench		<i>V_{SL_speed_MIN}</i> is the minimum selectable speed that allows Speed Limiter <i>V_{SL_speed_MAX}</i> is the maximum selectable speed that allows Speed Limiter <i>LVST</i> is the Last Valid Speed Threshold VehicleSpeed; This is the vehicle speed at time of SET/- or SET/+ button press during STORING mode. VehicleSetSpeed (VSS); This is a variable to determine the value that will be sent on CAN-C for display		All	CFTS_010_3 050 CFTS_010_3 051 CFTS_010_3 052
T011_56	1.1 Speed Limiter Command Management		Veh Sim Bench				All	CFTS_010_2 915 CFTS_010_2 195 CFTS_010_2 196 CFTS_010_2 197 CFTS_010_2 244
T011_57	1.1.1 Transition from INIT to OFF	Transition from INIT to OFF	Veh Sim Bench	CAN_A CQ DEV_S DT DIAG HWT	COND 1: Key-on, speed Limiter switched off COND 2: From previous condition start up the engine and act on gas pedal.	VER 1: CAN signal SpeedLimitSts = "Off" (110b) VER 2: As VER 1, no torque limitation is active.	All	CFTS_010_2 245 CFTS_010_2 246 CFTS_010_2 247
T011_58	1.1.2 Enable State		Veh Sim Bench				All	
T011_59	1.1.2.1 Entering from Off	Entering from Off	Veh Sim Bench	CAN_A CQ DEV_S DT	COND 1: Key-on, no fault present. Press and release Speed Limiter On/Off button	VER 1: CAN signal SpeedLimitSts = "Standby" (000b)	All	CFTS_010_2 258 CFTS_010_2 252

Usually this document is unique for architecture , because all the project developed under the same architecture can support the same features (functionalities) that can be enabled or not or simply calibrated differently for the specific application



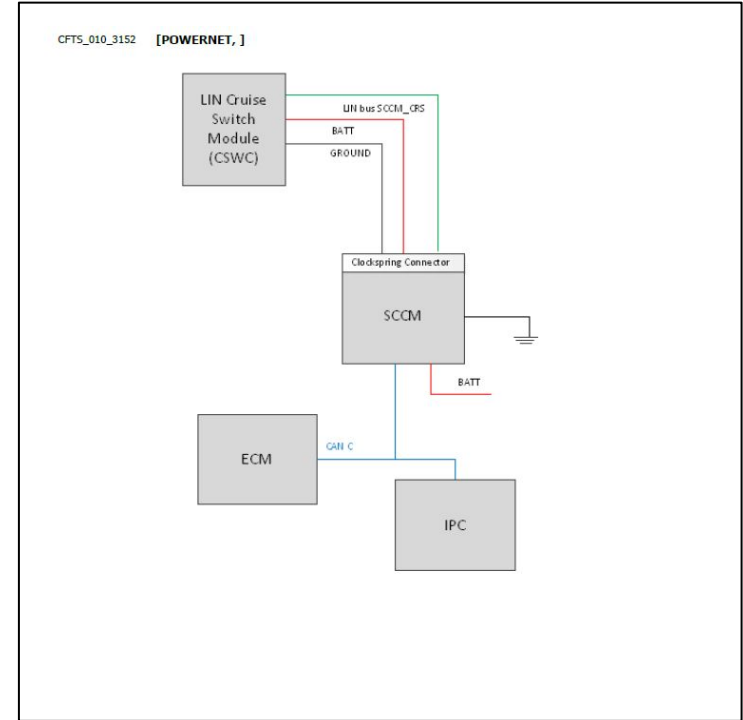
Validation Test example - ECM SpeedLimiter Overspeed management



Speed Limiter Overview

Whole ECM Speed Limiter management

The calculation of the speed setpoint is managed entirely by the engine control unit (that's why is defined as «whole», in some cases the speed limiter control is managed by two nodes): it receives signal from the steering wheel and (with the coordination of the ESP) defines the output speed limitation driver request , this value is then published on CAN and is displayed on the instrument cluster.



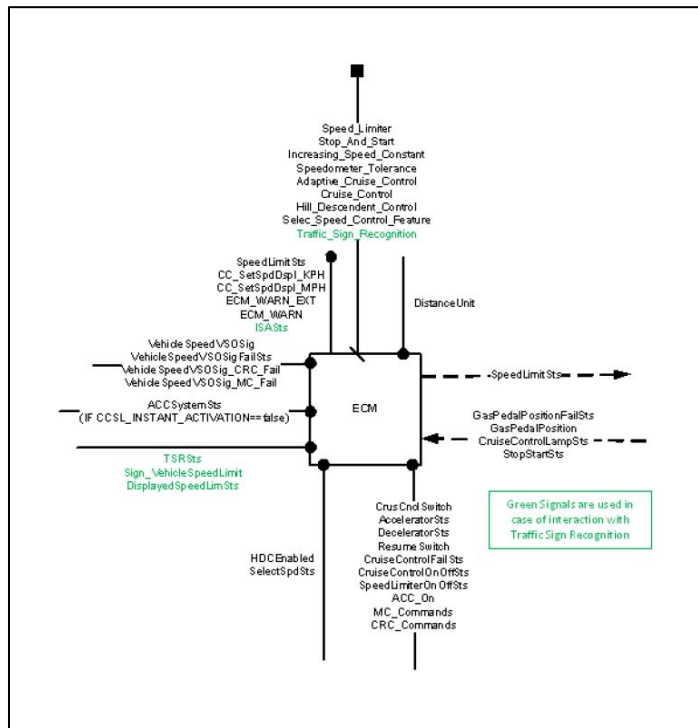


Speed Limiter Overview

Jeep Wrangler Speed Limiter ECUs interaction

The Speed Limiter is a high impact driver function whose main purpose is to limit the vehicle speed to a maximum threshold . Nowadays , this function is a standard equipment on almost every vehicle on the market and it usually implicates the interaction between at least 2 or more ECUs to elaborate the driver settings .

On PowerNet architecture we have the interaction between SCCM , IPC and ECM , the last one has the main duty to elaborate , once received , the driver command for the speed limitation and actuate the setpoint track .

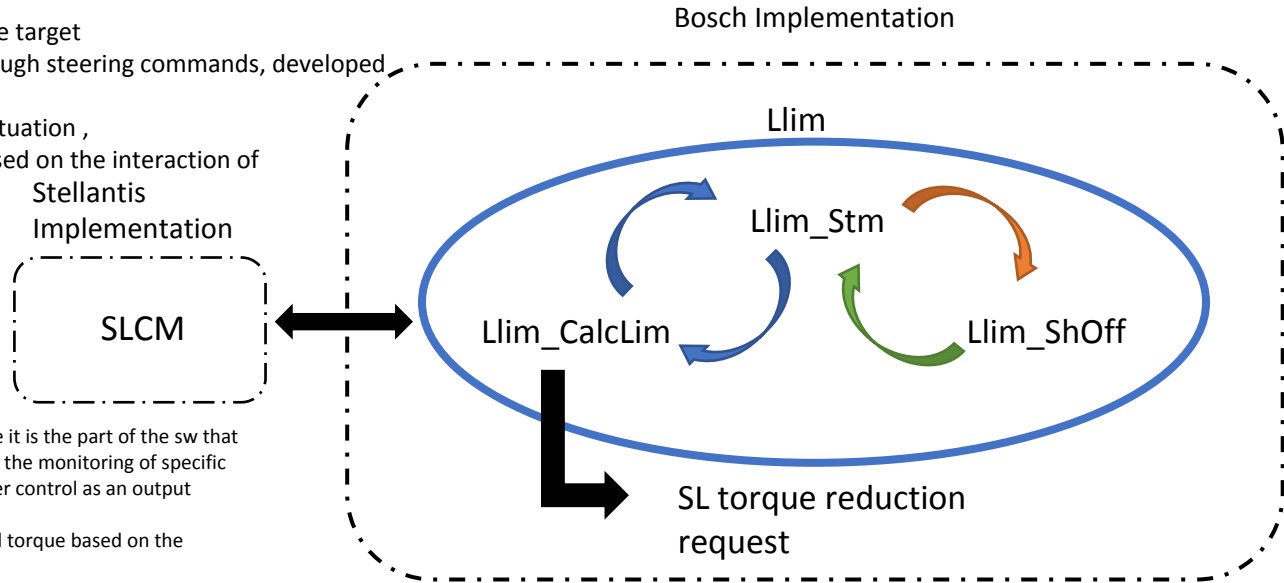




Speed Limiter Control Software implementation

On Jeep Wrangler project , the *Whole ECM Speed Limiter* control is a wrapped function and it is composed by two main blocks :

- SLCM : its function is to elaborate the target speed requested by driver through steering commands, developed internally by FCA
- Llim : manages the control speed actuation , developed by Bosch and it's based on the interaction of three subfunction.



Llim_Stm: Speed Limiter functionality status machine it is the part of the sw that recognizes the operational driving condition through the monitoring of specific inputs and manages consequentially the speed limiter control as an output state.

Llim_CalcLim: this module defines the output control torque based on the speed limiter status managed by the **Llim_Stm**.

Llim_ShOff: it necessary for the shutoff/inhibition of the SL control under proper condition defined by the status machine.



The diagram illustrates the control logic for the Limr SetSpd system, showing various states and transitions based on sensor inputs and internal logic. A green arrow labeled "Btn Activation" points to the "Enabled" state.

States and Transitions:

- NonActive:** Initial state. Transitions to **Enabled** (if `[[Diagc_SpdLimrFndFail-- 1]] && ...`) and **FAULT** (if `[[Diagc_SpdLimrFndFail-- 1]] && ...`).
- Enabled:** Central state. Transitions to **TapDown** (if `[[Limr_srtAmp-- 7]]`), **TapUp** (if `[[Limr_srtAmp-- 0]]`), **String** (if `[[Limr_chgDefInis-- 1]]`), **Increasing** (if `[[Limr_srtAmp-- 0]]`), and **Decreasing** (if `[[Limr_srtAmp-- 7]]`).
- TapDown:** State where the tap is down. Transitions to **Enabled** (if `[[Limr_srtAmp-- 7]]`).
- TapUp:** State where the tap is up. Transitions to **Enabled** (if `[[Limr_srtAmp-- 0]]`).
- String:** State where the string is active. Transitions to **Enabled** (if `[[Limr_chgDefInis-- 1]]`).
- Increasing:** State where the speed is increasing. Transitions to **Enabled** (if `[[Limr_srtAmp-- 0]]`).
- Decreasing:** State where the speed is decreasing. Transitions to **Enabled** (if `[[Limr_srtAmp-- 7]]`).
- FAULT:** State where a fault is detected. Transitions to **NonActive** (if `[[Diagc_SpdLimrFndFail-- 1]] && ...`).

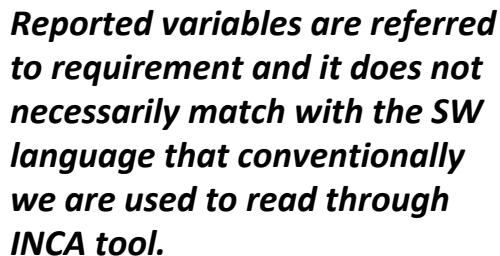
Logic Rules (Transitions):

- NonActive to Enabled:** `[[Diagc_SpdLimrFndFail-- 1]] && ...`
- Enabled to TapDown:** `[[Limr_srtAmp-- 7]]`
- Enabled to TapUp:** `[[Limr_srtAmp-- 0]]`
- Enabled to String:** `[[Limr_chgDefInis-- 1]]`
- Enabled to Increasing:** `[[Limr_srtAmp-- 0]]`
- Enabled to Decreasing:** `[[Limr_srtAmp-- 7]]`
- TapDown to Enabled:** `[[Limr_srtAmp-- 7]]`
- TapUp to Enabled:** `[[Limr_srtAmp-- 0]]`
- String to Enabled:** `[[Limr_chgDefInis-- 1]]`
- Increasing to Enabled:** `[[Limr_srtAmp-- 0]]`
- Decreasing to Enabled:** `[[Limr_srtAmp-- 7]]`
- FAULT to NonActive:** `[[Diagc_SpdLimrFndFail-- 1]] && ...`



The Overspeed is a specific condition of the state machine , this state is realized when the car is going faster than the setpoint track speed with the Speed Limiter engaged for an amount of time.

This important feature is regulated on the requirement specification.





SL Control Example : Checklist testcase representation

Previous requirement has been summarized into a test case Checklist description: performing the reported maneuver we are able to stress the system with the aim to reach the «overspeed» state (condition) and subsequently define if the system answer is in compliance with the spec (verification).

**Related CFTS
requirement ID**

ID	Test Function Name	Sub-Function	Test Env	Tools	Condition	Verification	E/E Architecture
T011_17	1.2.2.7 Transition from Active with ramp to Overspeed	Transition from Active with ramp to Overspeed	Veh Sim Bench	CAN_A CQ DEV_S DT DIAG HWT	COND 9: As COND 1 COND 10: From COND 9, keep the same vehicle speed for a time $t > t_{\text{timeout}}$ COND 1: key-on, engine running, Speed limiter switched ON - Increase gradually vehicle speed in order to have: $\text{VehicleSpeed} > v_{\text{lim}} + \text{thres_ovspd_in}$ - press resume button.	VER 9: As VER 1 VER 10: After t_{timeout} expires, - SpeedLimitSts = "Overspeed" (001b) - Internal vehicle speed limit variable v_{lim} is not updated - CC_SetSpdDspl_KPH = "v_{lim}" (km/h) - CC_SetSpdDsp_MPH = "v_{lim}" (MPH) - Torque limitation is active. - Vehicle speed is higher than the limit set point.	All

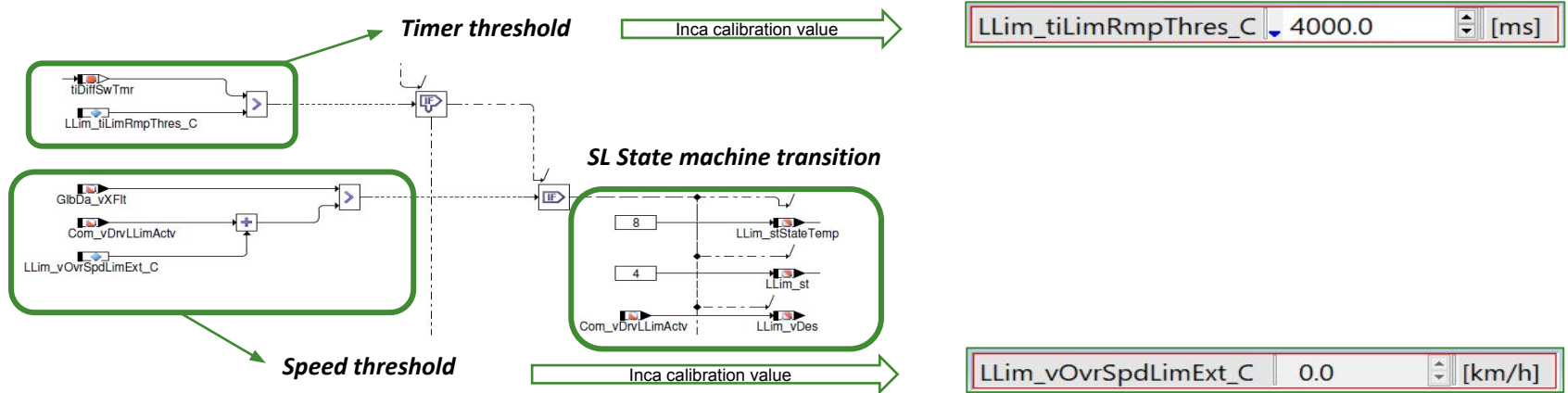
CFTS_010_2
321
CFTS_010_2
322



SL Control Example : I/O monitoring

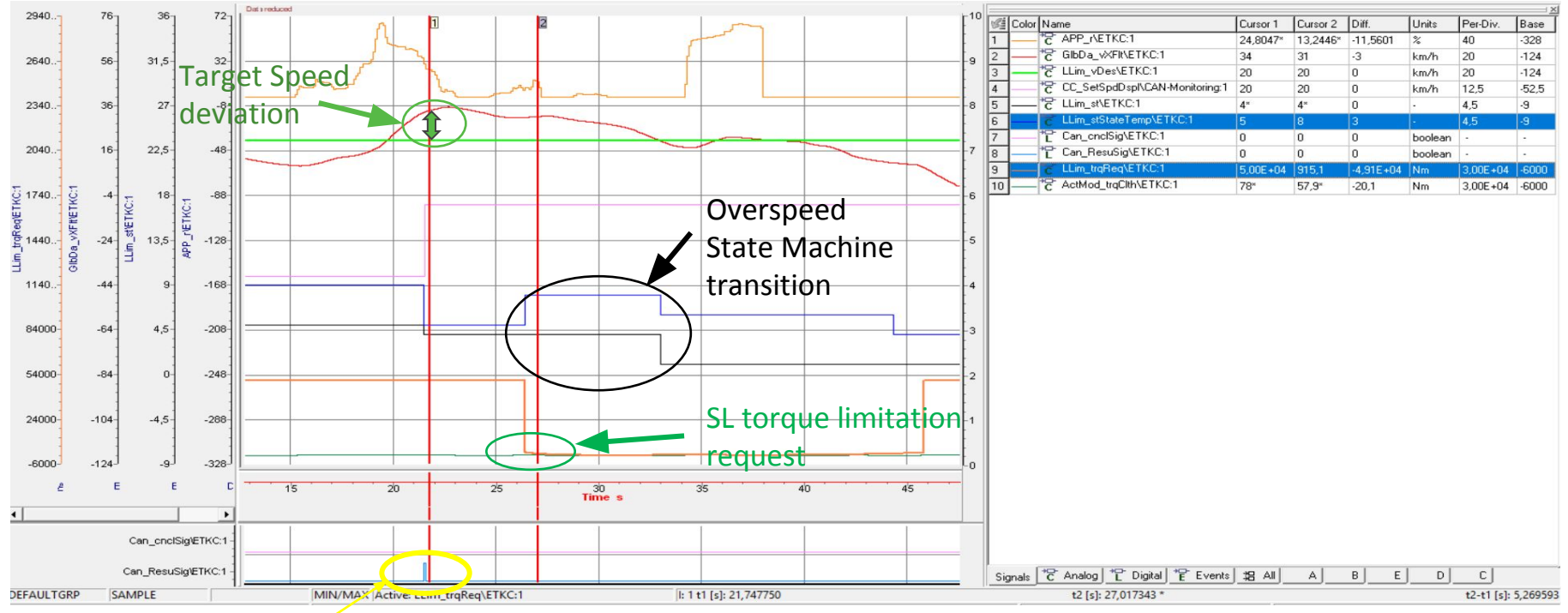
System state variables shall be acquired during test execution : in terms of input/output we shall monitor all the relevant sensed SW relevant (physical/logical value – vehicle speed and internal timers) and the system reaction (in this case the SL Status Machine evolution) involved with the functionality tested. On SW manual is reported the description of the Llim state transition with all the internal variables and parameters involved , this represents the interface between the user and the SW logic though INCA tool .

These are the threshold condition that must be achieved for the test execution





Measure analysis



Driver SL activation
command



Test Report

Test results are organized inside Report documentation.

Test function reference

Test result

Related result information

										1° LOOP													
Chap		Paragraph			ID	TEST NAME	REQUIRED	DETAILS	PRIORITY	SW VER	SW VAL	TESTED	TEST ENVIRONMEN	TEST RESULT	TESTED	TEST ENVIRONMEN	TEST RESULT	OPEN POINT Vif OPL / DOPL	AFFECTED Test	ACQUISITIONS	DCM	NOTE 1° LOOP	
1	2	2	6		T011_16	Transition from Active with ramp to Active with speed limiting	YES								YES	VP	OK			2017-11-29_CIL51SA56 X7_LLim	CIL51SA56 X7_LLim		
1	2	2	7		T011_17	Transition from Active with ramp to Overspeed	YES								YES	VP	OK			2017-11-29_CIL51SA56 X7_LLim	CIL51SA56 X7_LLim		
1	2	2	8		T011_26	Transition from Overspeed to Active with speed limiting	YES								YES	VP	OK			2017-11-29_CIL51SA56 X7_LLim	CIL51SA56 X7_LLim		
1	2	2	9		T011_27	Transition from Overspeed to Deactivated	YES								YES	VP	OK	V134		2017-11-29_CIL51SA56 X7_LLim	CIL51SA56 X7_LLim	After pressing Cancel button , Llim_vDes =	
1	2	2	10		T011_36	Transition from Enable to Active with speed limiting	YES								YES	VP	OK			2017-11-29_CIL51SA56 X7_LLim	CIL51SA56 X7_LLim		



Speed limiter NOK test example



SL Control Example: Interaction with S&S

Some tests are meant to study the interaction between different functions , so it's important to understand the coordination between the logics involved.

In this case we are going to analyze the behaviour of the SL after a S&S re crank.

T011_52	1.4.3 Stop and Start Interaction		Veh Sim Bench	CAN_A CQ DEV_S DT DIAG HWT	COND 1: With speed limiter Enabled but not engaged: - SpeedLimitSts = "Standby" (000b) - CC_SetSpdDspl_KPH = "v_lim" Fulfill all the conditions to perform an automatic stop. COND 2: From previous condition perform an automatic restart and bring the vehicle to a speed close to vehicle speed set point. Press resume button. COND 3: From previous condition press the brake pedal and perform an automatic stop. COND 4: From previous condition perform an automatic restart and bring the vehicle to a speed close to vehicle speed set point. Press resume button.	VER 1: S&S goes in STATUS 5 - SpeedLimitSts = "Standby" (000b) - CC_SetSpdDspl_KPH = "v_lim" Internal vehicle set speed is not updated. VER 2: Speed limiter starts to limit to the vehicle speed set v_lim - SpeedLimitSts = "Activated" (001b) - CC_SetSpdDspl_KPH = "v_lim" - Torque limitation active VER 3: S&S goes in STATUS 5 - SpeedLimitSts = "Standby" (000b) - CC_SetSpdDspl_KPH = "v_lim" Internal vehicle set speed is not updated. VER 4: Speed limiter starts to limit to the vehicle speed set v_lim - SpeedLimitSts = "Activated" (001b) - CC_SetSpdDspl_KPH = "v_lim" - Torque limitation active	All	CFTS_010_2 396
---------	---	--	---------------	---	--	---	-----	-------------------

6.11 Impact On Other ECM Functions

6.11.1 Interaction with other systems

6.11.1.1 Engine Stop&Start

CFTS_010_2396

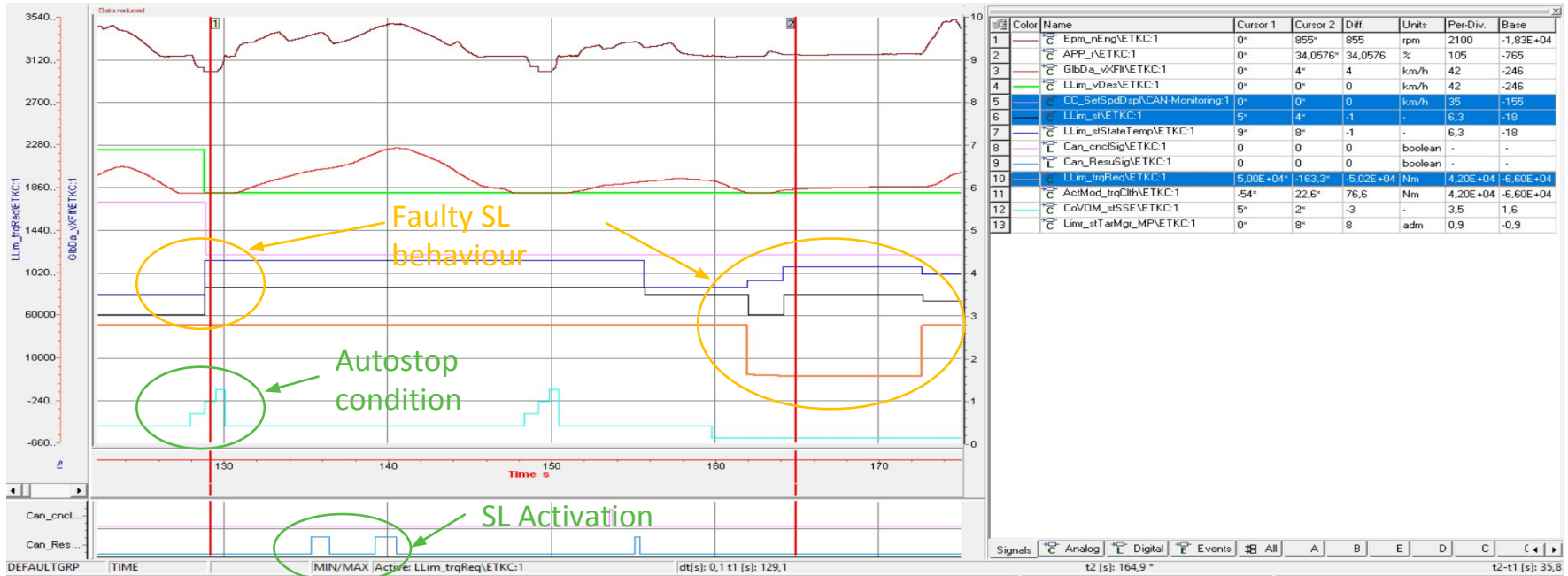
[Functional,]

- The ECM should not interrupt the CAN-C message transmission of the state of the system (ON/OFF) or cruise set speed during an ESS event.
- The ECM should not clear the state of the system (ON/OFF) or speed limit during an ESS event



Measure analysis for test report

In the measurement it is clearly visible that the requirement is being violated: the SL is wrongly turned off.





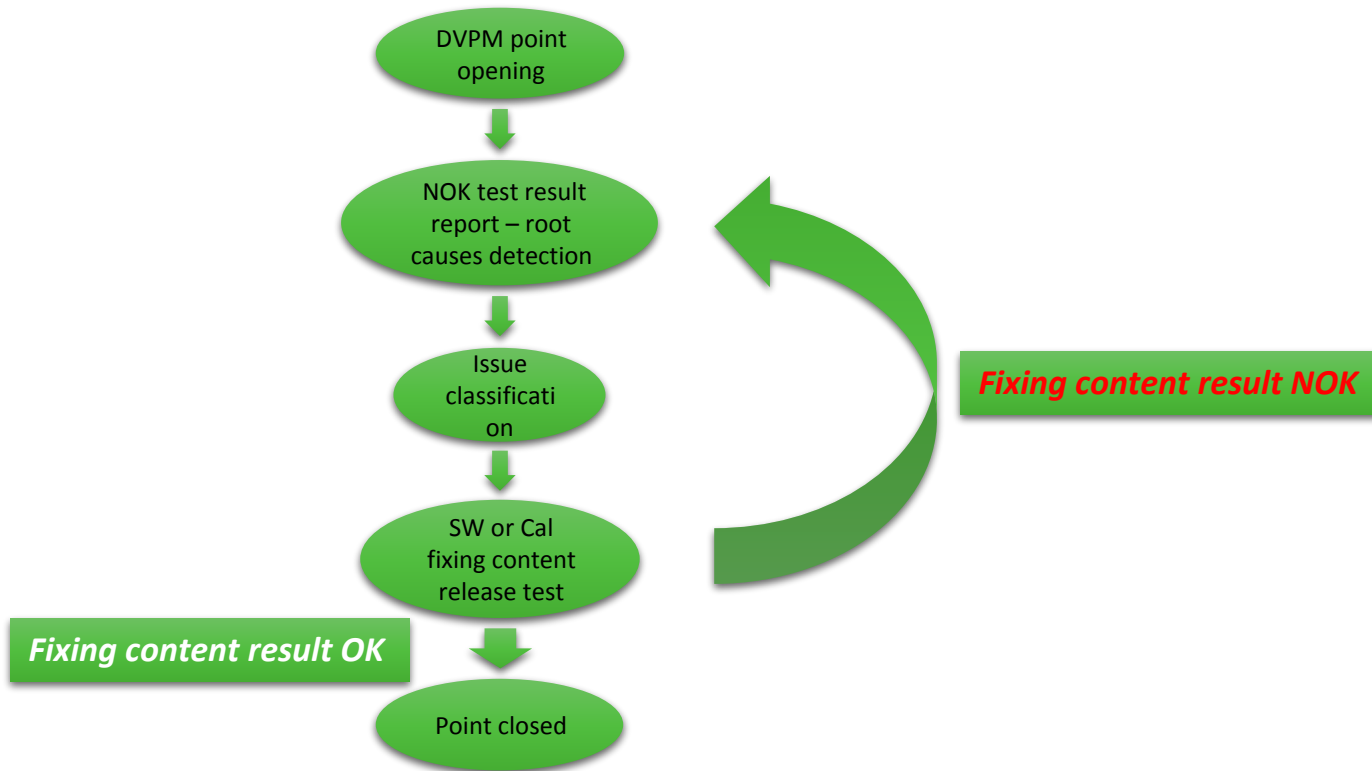
SW Issue detection

In this case the test result highlighted an undesired SW behaviour: the SL internal model was wrongly managing the State Machine transition, causing the limitation speed set at 0kmh after the autostop maneuver.

This is a SW bug since the problem is located inside the State Machine and it's hardcoded (no calibrations are involved).



Troubleshooting process





Validation process

All the NOK test will be reported on the DVPM document giving all the necessary description of the incorrect behaviour .

Through a macro implemented in the report document it is possible to define the progress of the tested functionality , thus it's possible to keep trace of the NOK tests and the anomalies that have to be fixed .

Progress (Ctrl+Q)						
Chapter	Test			Result		
	Required	Tested	%	OK	NOK	%
NEXTGEN	0	0	0,0%	0	0	0,0%
S&S	590	0	0,0%	0	0	0,0%
SAM	286	0	0,0%	0	0	0,0%
NGSL	0	0	0,0%	0	0	0,0%
SPEEDLIM	60	55	91,7%	37	18	32,7%
SCRoF	0	0	0,0%	0	0	0,0%
ECM-SCU	0	0	0,0%	0	0	0,0%
TOT	936	55	5,9%	0	0	0,0%

This information is useful to elaborate the global state of progress for the Activity



Vehicle test significance



Engine Cam-Backup strategy VF description

Some functionality tests keep their own significance only when tested in vehicle.

Crankshaft tone wheel back-up strategy:

Engine position is an essential entity; ECM must be able to recognize it in every operational condition. Several strategies directly descended from engine position & speed calculation (injection control, VVA, gear management, fuel pressure...etc) and contribute to the whole engine operation.

Whether Crankshaft tone wheel signal is corrupted or not available at all, ECM shall be able to calculate the right position using Camshaft tone wheel input value in order to define the current engine speed. This recovery action is implemented for safety reasons; it guarantees the engine to be operative even in degraded states.

Such a strategy is a required feature which is usually tested during Engine Position Management validation.



Engine Cam-Backup strategy VF description

32.11.2.1 Engine Speed Calculation

Engine speed/RPM shall be calculated internally in the ECU from crank sensor teeth period (tooth to tooth time) as soon as valid crank teeth information is available AND while valid top dead center information is not yet available.

Top Dead Center Positions shall be considered valid for computing rpm once the crank position bus is locked and the two TDC positions have been traversed.

IF valid TDC information is available, engine speed/RPM shall be calculated internally in the ECU from two TDCs, and no longer from crank sensor teeth period.

Engine speed/RPM shall also be calculated from the cam position sensor(s) IF the cam position sensor(s) are locked AND two TDC positions have been traversed.

IF the crank sensor is not faulted/failed, Engine speed/RPM shall be reported over CAN from the valid crank sensor.

IF the crank sensor is faulted/failed AND the cam sensor is not faulted/failed, the engine speed/RPM shall be reported over CAN from the valid cam sensor signal

Engine speed will be calculated down to the point where the active/valid sensor (crank or cam) becomes unlocked, and at the point the RPM signal is ramped down to "0".

Engine speed will be calculated up to 8160 RPM and sent on the bus in ECM_A1.EngRPM

Engine speed shall be sent on the bus in ECM_A1.EngRPM

Cam BackUp
strategy



BackUp SW management

ECM is able to detect every tooth transition and recognizes crankshaft edges; tooth counting is buffered and internally processed in order to obtain the segment period; this information defines the engine speed. If a single or multiple tooth is not available (where expected) the ECM recognizes the issue; consequently, the injection is no more applied and the engine comes to stop.

After 7 crankshaft revolutions with systematic missing tooth, Crankshaft source is recognized as faulty (calibratable issue debounce). As a consequence, CAM BackUp strategy is applied (in this scenario a Diagnostic Fault is expected).

In the same way, if Crankshaft signal is not available the ECM will adopt a filtering phase waiting for 7 crankshaft revolutions - calculated by Camshaft signal (Crankshaft & Camshaft position are strictly related) - before applying Cam-BackUp strategy.

When BackUp strategy is applied, if the actual engine speed (calculated by CamShaft signal) is greater than a fixed threshold (ex. 500 RPM), the ECM will pilot the injection in order to keep the idle engine speed, otherwise the engine will be shutted down.



Test

Let's consider the simplest scenario: Cam Backup strategy in case of Crankshaft LoS (no digital sensed value by ECM).

Which parameters are we looking for in order to validate the strategy?

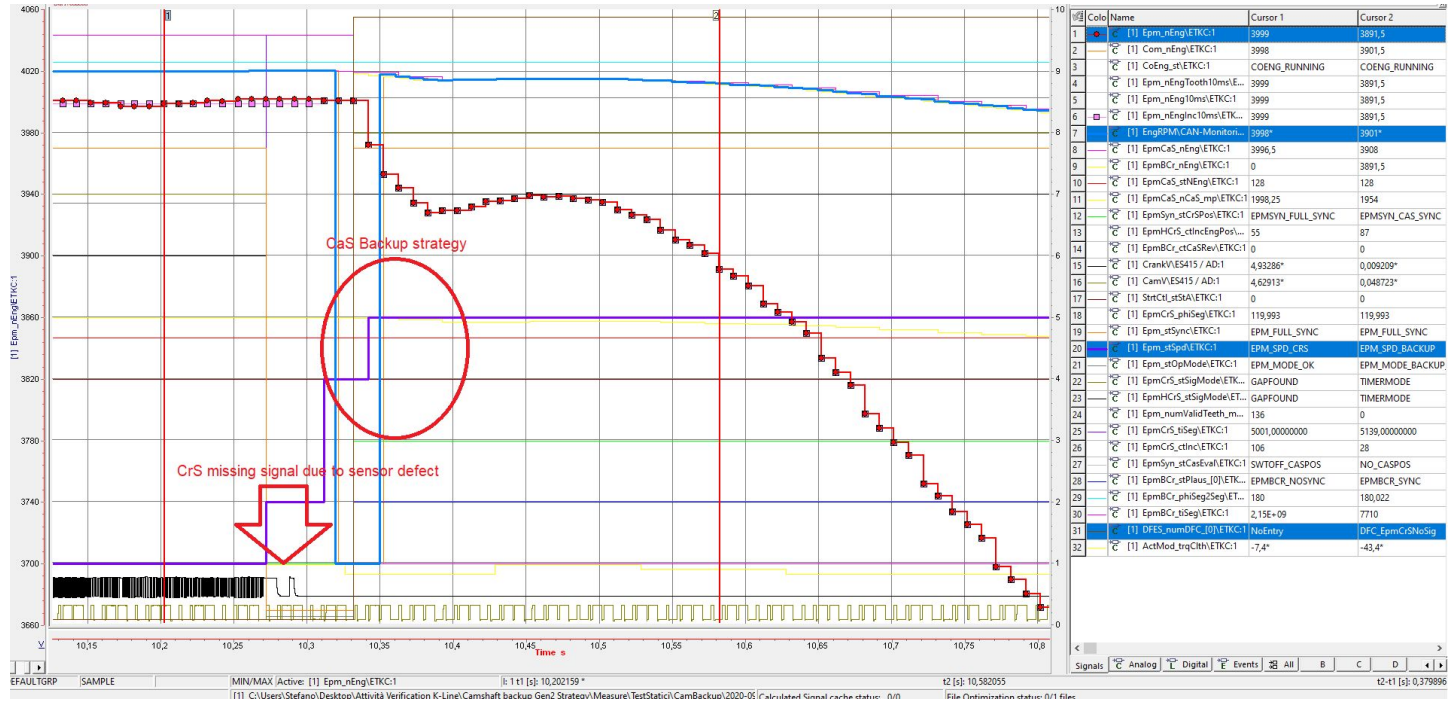
How do we set up the test?

How do we execute the test?

Why does this test is more significant in vehicle rather than in HiL environment?



Test Result





Conclusions: Environmental significance

When CrankShaft LoS has been performed in idle conditions, two different results stand out from different testing environment.

When HiL-tested, Cam BackUp strategy correctly worked with a 8/10 successful ratio of engine idling recovering. This means that for 8 cases in 10 if the Crankshaft signal was suddenly missing - in idle conditions- the ECM was able to correctly adopt backup strategy in order to keep the engine “alive”.

On the contrary, when vehicle-tested, the ratio dropped down to approximately 5/10.

This other ratio was due to actual engine physical behaviour, different from the simulated one: some model dynamics (such as frictions losses, mechanical inertia, etc...) were not reliable and it had a big impact on this kind of analysis.

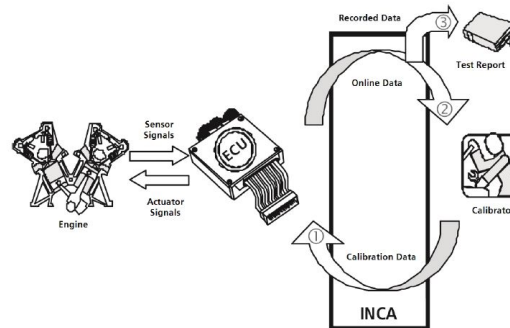
This example shows how test significance depends on the performing environment and, moreover, not every test can be reproduced in a simulated one.



Thank you

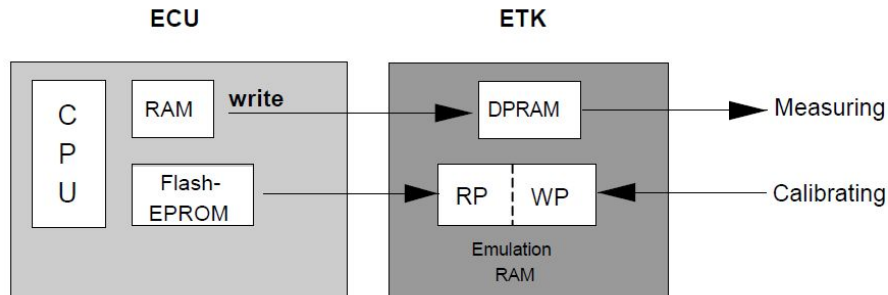
Appendix

- Calibration – Nowadays, a large variety of engine vehicle hardware layout are under the domain of a common compatible electronic unit. This means that in some case the engine electronic architecture is likely the same for two or more different vehicle models (this in particular for the models of a specific manufacturer), thus same engine SW might be applied on: different engines version can be managed by a same electronic unit with a common code. How can we let a 1.6l 120hp manual transmission and a 2.0l 180hp automatic transmission be managed by the same software application? The answer is calibration. The calibration process has the purpose to adapt the applicative software for that specific system. In terms of control, it allows the SW to work correctly using the polyvalent controls logic coded tooled to that engine application. In the end, the software is able to manage either 1.6 manual and 2.0 automatic transmission, through calibration it is possible to define the control output.



Appendix

- ETK – Emulator-Tastkopf (emulator test probe). Specific hardware (usually integrated) device, so called «development ECUs» that allows an user to analyze and modify the control logic supported by the SW. ETK interface is supported by ETAS tools and devices. Tools such INCA allows an engineer to monitor real time internal sw (high level entity) values and modify through calibration control logic. An emulator test probe (ETK) is installed in the control unit for the parallel application. The ETK is connected to all bus lines of the control unit processor and replaces either the control unit program and the calibration data or the calibration data only – depending on the control unit. Consequently, the calibration system can change control parameters online. Furthermore, the ETK provides RAM (socalled DPR = Dual Port RAM) to transfer measured data from the control unit to the calibration system. The ETK is the control unit-specific component of the memory emulation.



Appendix

- CCP/XCP - CAN Calibration Protocol/ eXtended Calibration Protocol. Both protocols are finalized to the calibration use through conventional interfaces different from ETK, they enable read and write access to variables and memory contents of microcontroller systems at runtime (even sw flashing). CCP is a protocol standardized on the CAN bus. XCP uses more interfaces options based on different communication protocols such as CAN, FlexRay, Ethernet or USB.

CONTACTS

Stefano Misuraca
stefano.misuraca@kineton.it

Via E. Gianturco, 23
(presso complesso Mecfond)
80146 Napoli

www.kineton.it

